Name - Jayesh Deshmukh
Class - D15C
Roll Number - 57

EXPERIMENT 3 : Implement K-Nearest Neighbors (KNN) and Evaluate Model Performance

---

1. Dataset Source:

Dataset Name: Pima Indians Diabetes Dataset
Source: Kaggle
Link: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database

The dataset contains medical diagnostic measurements used to predict whether a patient has diabetes.

---

2. Dataset Description:

The Diabetes Dataset is used to demonstrate **K-Nearest Neighbors (KNN)** for binary classification.

This dataset contains medical predictor variables and one target variable indicating whether the patient has diabetes.

• **Size:** 768 samples × 9 attributes

• **Target Variable:**
Outcome
0 → No Diabetes
1 → Diabetes

• **Input Features:**

1. Pregnancies
2. Glucose
3. BloodPressure
4. SkinThickness
5. Insulin
6. BMI
7. DiabetesPedigreeFunction
8. Age

• **Dataset Characteristics:**
● Fully numerical features
● No categorical encoding required
● Requires feature scaling
● Suitable for binary classification

---

3. Mathematical Formulation:

K-Nearest Neighbors (KNN):

KNN is a supervised learning algorithm that classifies a new data point based on the majority class of its K nearest neighbors.

Distance Calculation (Euclidean Distance):

$d(x,y)=\sum i=1n(xi-yi)2$d(x,y)=i=1∑n(xi−yi)2

Where:

- xx = new data point
- yy = training data point
- nn = number of features

Prediction Rule:

Predicted Class=Majority Vote of K Nearest NeighborsPredicted Class=Majority Vote of K Nearest Neighbors

---

4. Algorithm Limitations:

1. Sensitive to feature scaling.
2. Computationally expensive for large datasets.
3. Performance depends heavily on choice of K.
4. Sensitive to noisy data and outliers.

Not suitable when:
• Dataset is extremely large.
• Features are not properly normalized.

---

5. Methodology / Workflow:

Step 1: Data Collection

• Dataset downloaded from Kaggle.

Step 2: Data Preprocessing

• Load dataset using Pandas
• Check for null values
• Separate features and target variable
• Perform train-test split (80% training, 20% testing)
• Apply StandardScaler for feature normalization

Step 3: Model Training

• Train K-Nearest Neighbors Classifier
• Select optimal value of K

Step 4: Model Evaluation

• Predict on test data
• Calculate Accuracy
• Generate Confusion Matrix
• Generate Classification Report

---

6. Performance Analysis:

Evaluation Metrics Used:

• Accuracy
• Precision
• Recall
• F1-Score
• Confusion Matrix

Typical Results:

KNN (K=5):
Accuracy ≈ 0.75 – 0.80

Interpretation:

• Model achieves moderate classification accuracy.
• Proper scaling improves performance significantly.
• Optimal K value reduces error rate.
• KNN is simple yet effective for medical diagnosis problems.

---

7. Hyperparameter Tuning:

Hyperparameters Tuned:

• n_neighbors (K value)
• metric (distance metric)

Tuning Method:

Manual testing or GridSearchCV

Example:
Tested K values = [1 to 20]

Impact of Tuning:

Before tuning:
Accuracy ≈ 0.74

After tuning:
Accuracy ≈ 0.79

Code & Output-# ==========

```
# Experiment: K-Nearest Neighbors (KNN)

# Dataset: Diabetes Dataset

# ==========



# Step 1: Import Libraries

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns



from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report



# Step 2: Load Dataset

df = pd.read_csv("diabetes.csv")    # If using Colab upload first

# If using provided path:

# df = pd.read_csv("/mnt/data/diabetes.csv")



print("First 5 Rows:")

print(df.head())
```

```
print("\nDataset Info:")
print(df.info())

# Step 3: Define Features and Target
X = df.drop("Outcome", axis=1)    # Features
y = df["Outcome"]                 # Target (0 = No Diabetes, 1 = Diabetes)

# Step 4: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 5: Feature Scaling (IMPORTANT for KNN)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 6: Apply KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# Step 7: Predictions
```

```
y_pred = knn.predict(X_test)



# Step 8: Model Evaluation

print("\n===== KNN Model Performance =====")



accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)



print("\nConfusion Matrix:")

cm = confusion_matrix(y_test, y_pred)

print(cm)



print("\nClassification Report:")

print(classification_report(y_test, y_pred))



# Step 9: Confusion Matrix Visualization

plt.figure(figsize=(6,5))

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")

plt.title("Confusion Matrix")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```

```python
# Step 10: Finding Best K Value

error_rates = []



for i in range(1, 21):

    knn = KNeighborsClassifier(n_neighbors=i)

    knn.fit(X_train, y_train)

    pred_i = knn.predict(X_test)

    error_rates.append(np.mean(pred_i != y_test))



plt.figure(figsize=(8,5))

plt.plot(range(1, 21), error_rates, marker='o')

plt.title("Error Rate vs K Value")

plt.xlabel("K Value")

plt.ylabel("Error Rate")

plt.show()
```

```
First 5 Rows:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None

===== KNN Model Performance =====
Accuracy: 0.6948051948051948
```

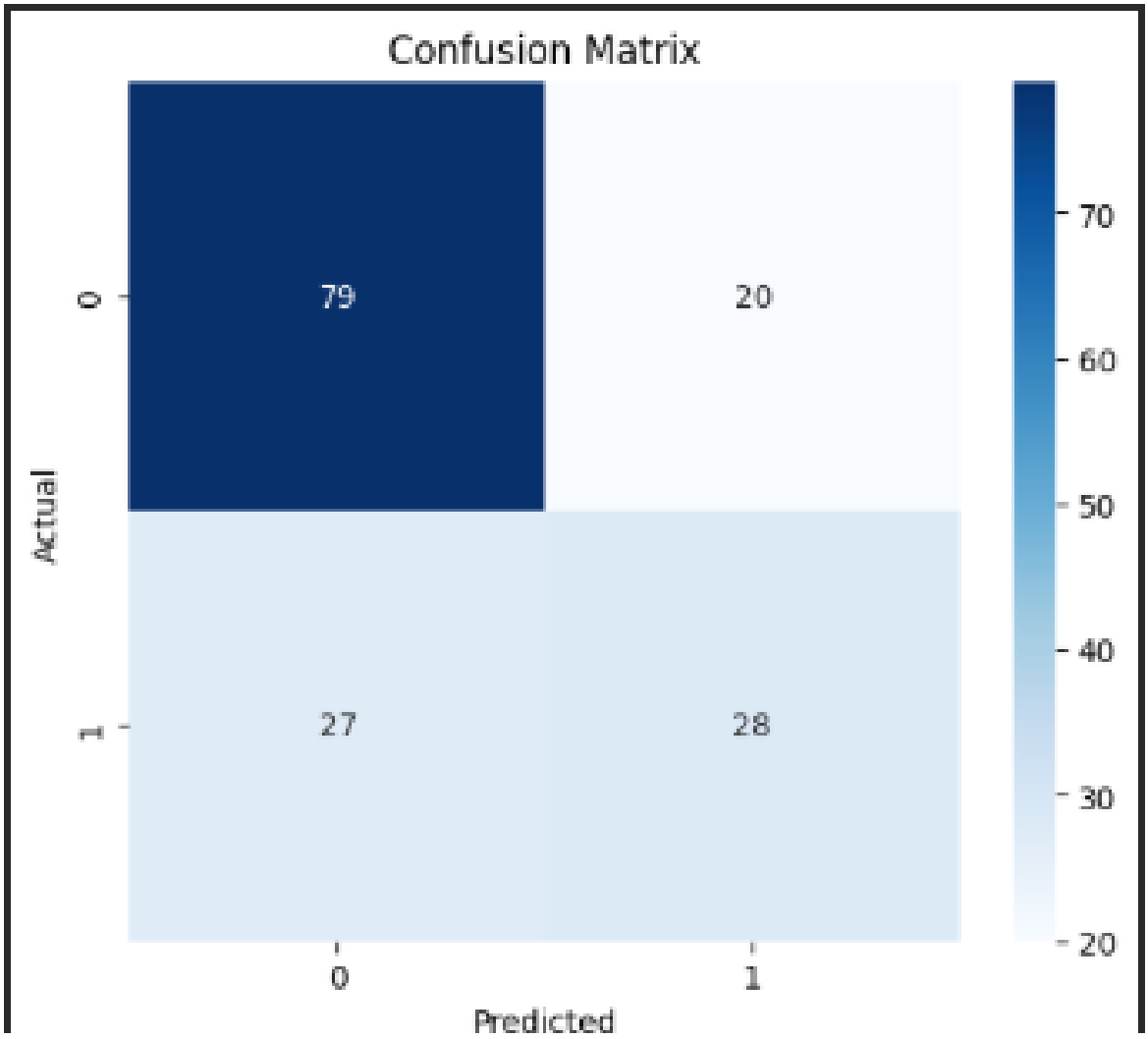
Toggle Gemini

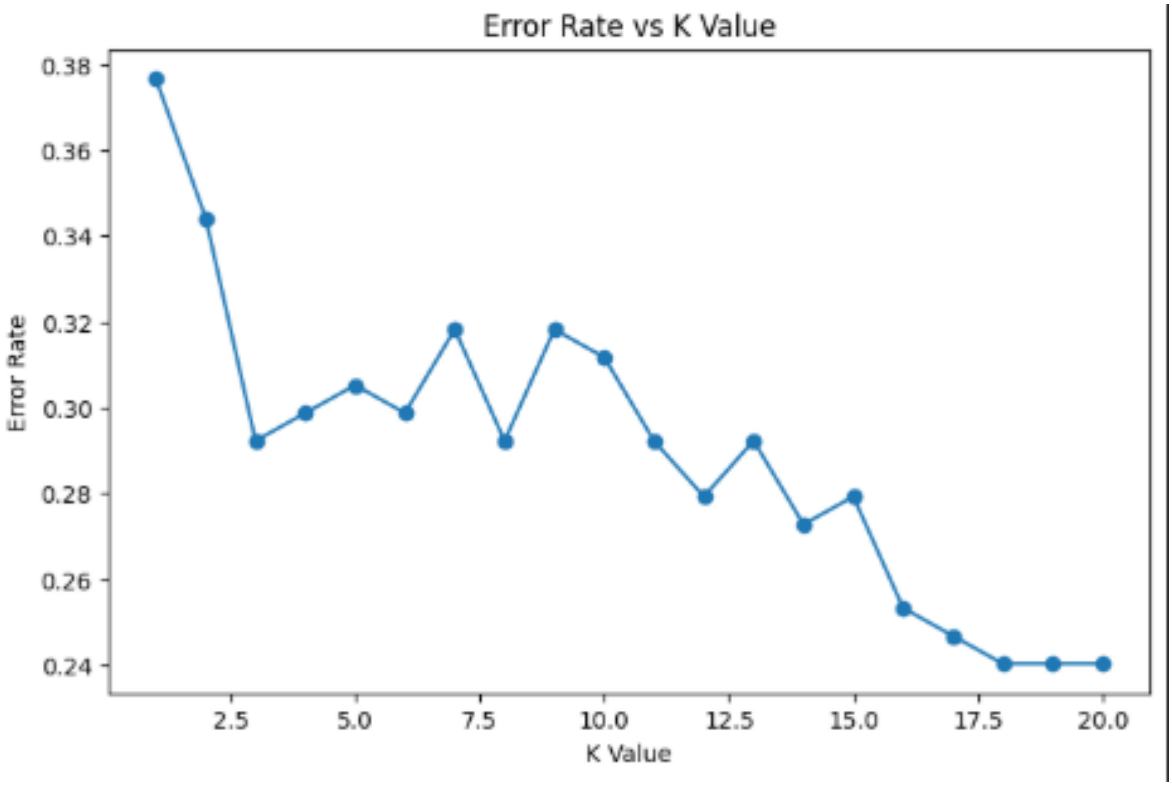
```
Confusion Matrix:
[[79 20]
 [27 28]]

Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.80      0.77        99
           1       0.58      0.51      0.54        55

    accuracy                           0.69       154
   macro avg       0.66      0.65      0.66       154
weighted avg       0.69      0.69      0.69       154
```


Confusion Matrix
79
20
27
28
0
1
Actual
0
1
Predicted
70
60
50
40
30
20


Error Rate vs K Value
Error Rate
0.38
0.36
0.34
0.32
0.30
0.28
0.26
0.24
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
K Value